向量加法执行过程分析

一、测试程序与指令

测试使用 tests/vadd/main.c 测试昆明湖 V2 的向量加法。程序准备两组各 16 个 int8 元素的输入，开启向量状态后执行配置、加载、加法和存储。关键指令顺序如下，地址操作数在 C 内联汇编中由编译器提供。

```
vsetivli zero,16,e8,m1,tu,mu    设置 16 个 8 位活动元素
vle8.v v8,(输入 A 地址)          装入第一组输入
vle8.v v9,(输入 B 地址)          装入第二组输入
vadd.vv v10,v8,v9                对应元素相加
vse8.v v10,(结果地址)            将结果写回数组
```

配置 VLEN 为 128，SEW 为 8，LMUL 为 1，一个向量寄存器容纳 16 个 8 位元素。本例 VL 为 16，全部元素参与运算。vadd.vv 使用无掩码编码，vta/vma 设置不会改变本例全部活动元素的计算。

反汇编给出的目标 PC 为 0x8000015e，指令字为 0x02848557。源码标签 kks_vadd_begin、反汇编和波形中的 PC 用于定位这条指令。

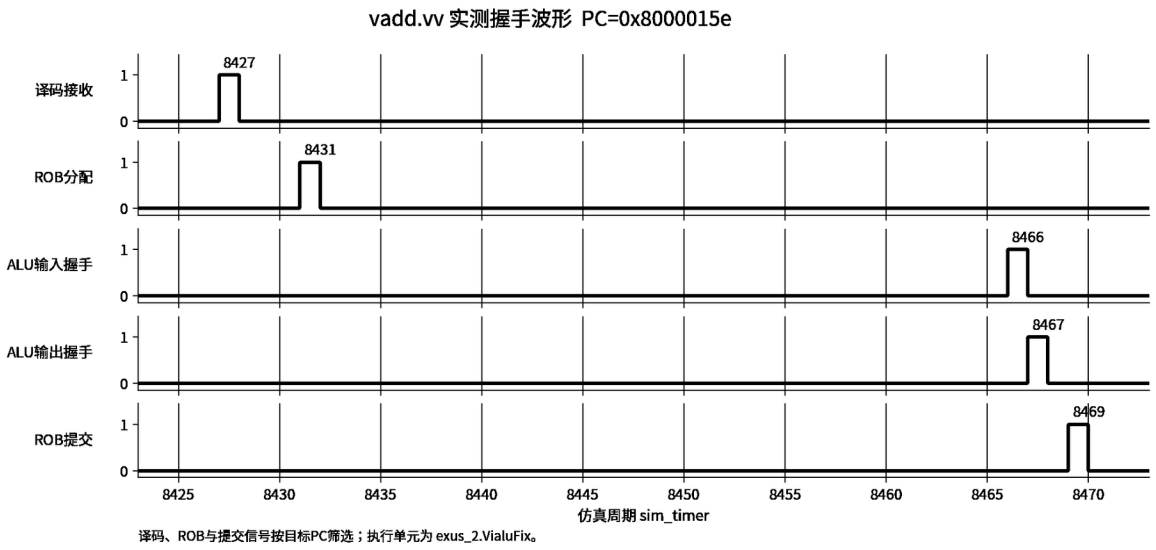
二、波形观察方法

仿真时使用--dump-wave 保存 VCD。提取脚本在每个时间点完成信号更新后采样时钟高电平，用 ready 和 valid 检查执行接口握手，用目标 PC 筛选译码、ROB 入队和提交记录。

下面的周期来自 VCD 中的 difftest_timer。表格采用 sim_timer 计数器的值。GTKWave 标记按 VCD 中的采样位置设置。

观察位置	**sim_timer**	**本次观察**
译码接收	8427	第 0 槽 valid 和 ready 同时有效，PC=0x8000015e，指令字 0x02848557
ROB 分配	8431	第 0 槽出现同一 PC 与指令字的入队记录
ALU 输入	8466	exus_2.VialuFix 输入 valid 和 ready 同时有效，两个 128 位源操作数到达
ALU 输出	8467	同一执行单元输出握手，128 位结果可

观察位置	sim_timer	本次观察
		用
ROB 提交	8469	commitValid 与 isCommit 有效，提交 PC=0x8000015e



图中的译码、ROB 和提交信号按目标 PC 筛选。

三、指令经过各部分的过程

前端取得指令后，译码识别向量加法，并建立向量源、目的寄存器和执行单元信息。重命名和 ROB 记录程序顺序及目的寄存器关系。程序中的两次向量加载为 v8、v9 提供数据；源操作数就绪后，指令发射到向量整数 ALU。

在 8427 拍观察到译码接收。8431 拍，目标指令进入 ROB。到 8466 拍，VialuFix 输入接口收到两个完整向量。这个接口到达前的 35 拍间隔包含源操作数等待与调度，本次没有逐项分解这些等待的原因。

ALU 按 8 位元素分组相加，在 8467 拍给出结果。输入到输出相差 1 拍。结果进入向量写回路径，后续存储指令读取结果。8469 拍，ROB 提交该条 vadd。这里直接测量了五个观察点；取指、物理寄存器分配和旁路的具体周期没有单独采样。

四、操作数与结果

ALU 输入接口记录的两个 128 位操作数为：

```
src0 = 0x04030201d600ff0104030201281e140a
src1 = 0x0b0a09082a00807ffcfdfeff04030201
```

低地址元素位于向量的低 8 位。两源端口的排列由执行接口定义；逐元素相加得到：

```
result = 0x0f0d0b0900007f80000000002c21160b
```

按低字节到高字节拆开，结果为：

```
0b 16 21 2c 00 00 00 00 80 7f 00 00 09 0b 0d 0f
```

元素编号	输入 A	输入 B	数学和	存入的 8 位结果
0	1	10	11	0x0b
8	127	1	128	0x80
9	-128	-1	-129	0x7f
15	11	4	15	0x0f

普通 vadd 按 8 位宽度保留低位，未进行饱和处理。C 循环按相同的模 256 规则检查全部 16 个存储结果。本次程序输出 VADD PASS，随后在 PC=0x800001c6 处到达 GOOD TRAP，DiffTest 未报告架构状态差异。

五、构建与波形查看命令

在仓库根目录执行：

```
source scripts/env.sh
bash scripts/build_app.sh vadd
rg -n -A 3 -B 3 kks_vadd_begin build/apps/vadd.disasm
bash scripts/run_emu.sh vadd baseline diff \
  --dump-wave --wave-path "$KKS_ROOT/build/stage1-vadd.vcd"
python3 scripts/extract_vadd_wave.py build/stage1-vadd.vcd \
  --pc 0x8000015e --output build/stage1-wave
gtkwave build/stage1-wave/vadd-selected.vcd
```

反汇编中目标 PC 为 0x8000015e。更换程序或工具链后，以新反汇编的 PC 定位指令。GTKWave 中将目标 PC、指令字、译码 valid/ready、ROB 入队、ALU 输入输出及提交信号加入视图，观察各接口是否有效。

六、测试程序

```
#include "kks.h"

static const int8_t lhs[16] __attribute__((aligned(16))) = {
  1, 2, 3, 4, -1, -2, -3, -4, 127, -128, 0, 42, 8, 9, 10, 11
};
static const int8_t rhs[16] __attribute__((aligned(16))) = {
  10, 20, 30, 40, 1, 2, 3, 4, 1, -1, 0, -42, 1, 2, 3, 4
};
static uint8_t result[16] __attribute__((aligned(16)));

int main(void) {
  kks_enable_vector();
  __asm__ volatile(
```

```
    "vsetivli zero, 16, e8, m1, tu, mu\n\t"
    "vle8.v v8, (%0)\n\t"
    "vle8.v v9, (%1)\n\t"
    ".global kks_vadd_begin\nkks_vadd_begin:\n\t"
    "vadd.vv v10, v8, v9\n\t"
    ".global kks_vadd_end\nkks_vadd_end:\n\t"
    "vse8.v v10, (%2)"
    : : "r"(lhs), "r"(rhs), "r"(result)
    : "v8", "v9", "v10", "memory");
  for (int i = 0; i < 16; ++i) {
    const uint8_t expected = (uint8_t)((uint8_t)lhs[i] + (uint8_t)rhs[i]);
    if (result[i] != expected) {
      printf("VADD FAIL lane=%d got=%d expected=%d\n", i, result[i], expected);
      return 1;
    }
  }
  printf("VADD PASS lanes=16, including int8 wraparound\n");
  return 0;
}
```